

OC3-3: Integrity Architecture, RBAC Enforcement, and Client-Side Attack Prevention

Table: OC3-3 Evidence Summary

Claim	Code / File	Verification
Bybit-class UI attack prevention	ui-integrity.ts — 4-layer hash integrity check	frontend/frontend/src/lib/ui-integrity.ts
Six-tier RBAC enforcement	rbac_config.json shared across Flutter + Next.js	frontend/shared/rbac_config.json
War Room dashboard (R3–R6)	Next.js App Router; live alerts, sanctions panel	frontend/frontend/src/app/
Flutter Focus Mode (R1–R2)	Riverpod + go_router; consumer wallet UI	frontend/amttp_app/lib/
Model explainability / audit	Decision factors + weights per CRITICAL alert	backend/compliance-service/explainability.py
Sole-authored full-stack UI	Flutter (6 platforms) + Next.js dashboard	github.com/segetii/AMTTP/frontend/

Table: OC3-3 Evidence Summary

1. Executive Summary & Core Threat Model

I engineered and implemented an enterprise compliance dashboard and mobile interface featuring cryptographic client-side UI integrity verification, a shared six-tier Role-Based Access Control (RBAC) matrix, and native AI explainability views. The front-end applications were developed as an integrated ecosystem to mitigate client-side injection attacks. In standard Web3 interfaces, cross-site scripting (XSS) or malicious browser extensions can alter the visual elements of an administrative approval screen, tricking an operator into signing a malicious payload. The architecture implemented within the AMTTP system introduces runtime cryptographic fingerprinting across all interface components, ensuring any unauthorised DOM modification instantly invalidates the execution context before a private-key signature can be broadcast.

2. Cryptographic UI Integrity Implementation (ui-integrity.ts)

The file ui-integrity.ts provides four layered defences: (1) every screen component carries a unique cryptographic fingerprint; (2) approvals are signed against actual transaction data, not the displayed text; (3) the server independently validates the fingerprint before acting; and (4) a final confirmation is shown in a secure, isolated context. If an attacker replaces any part of the interface, the fingerprints will not match and the transaction is blocked before any approval can be given.

```
TS ui-integrity.ts X
1  /**
2   * AMTTP UI Integrity Protection
3   *
4   * Prevents Bybit-style attacks where attackers manipulate the UI/signing logic
5   * to trick users into approving malicious transactions.
6   *
7   * Protection layers:
8   * 1. Component hash verification - detect DOM/logic tampering
9   * 2. Transaction intent signing - sign actual intent, not displayed data
10  * 3. Server-side hash validation - verify UI hasn't been modified
11  * 4. Visual confirmation layer - show hash-verified data in isolated context
12  */
13
14  // =====
15  // CONSTANTS
16  // =====
17
18  const INTEGRITY_VERSION = "1.0.0";
19
20  // Known good hashes (should be loaded from server in production)
21  // These are updated when legitimate deployments happen
22  const TRUSTED_COMPONENT_HASHES: Record<string, string> = {};
23
24  // =====
25  // CRYPTO UTILITIES
26  // =====
27
28  /**
29   * Generate SHA-256 hash of content
30   */
31  async function sha256(content: string): Promise<string> {
32    const encoder = new TextEncoder();
```

Figure 1: ui-integrity.ts lines 1–32 — header names the Bybit attack vector, lists four prevention layers, implements SHA-256 hashing via Web Crypto API. Any interface substitution breaks every hash.

```
159
160 export interface ComponentIntegrity {
161   componentId: string;
162   sourceHash: string;
163   domHash: string;
164   eventHandlersHash: string;
165   combinedHash: string;
166   verified: boolean;
167   timestamp: number;
168 }
169
170 /**
171  * Capture the integrity state of a UI component
172  */
173 export async function captureComponentIntegrity(
174   componentId: string,
175   componentElement: HTMLElement | null,
176   sourceCode?: string
177 ): Promise<ComponentIntegrity> {
178   const timestamp = Date.now();
179
180   // Hash the source code (if available)
181   const sourceHash = sourceCode ? await sha256(sourceCode) : "not-available";
182
183   // Hash the current DOM structure
184   let domHash = "not-available";
185   if (componentElement) {
186     // Remove dynamic content, keep structure
187     const structure = extractDOMStructure(componentElement);
188     domHash = await sha256(structure);
189   }
190 }
```

Figure 2: ui-integrity.ts lines 160–190 — four independent hashes (sourceHash, domHash, eventHandlersHash, combinedHash) per UI component. Hashing source code and live DOM independently detects the Bybit attack pattern.

3. Distributed Role-Based Access Control (RBAC) Framework

Role	Label	Interface	Key Capabilities
R1	End User	Flutter (Focus Mode)	Own transactions only
R2	PEP End User	Flutter (Focus Mode)	Enhanced monitoring, restricted view
R3	Compliance Officer	Next.js (War Room)	All transactions, report generation
R4	Risk Analyst	Next.js (War Room)	Detection studio, policy editing
R5	Administrator	Next.js (War Room)	User management, enforcement actions
R6	Super Admin	Next.js (War Room)	Emergency override, full audit access

The governance engine enforces a rigorous separation of operational duties across six discrete administrative tiers. Server-Side Attestation: while roles dictate the presentation views within the Next.js and Flutter front-ends, access permissions are verified strictly at the backend API gateway layer via tokenised claims. Bypassing or modifying client-side visibility flags yields zero access, as unauthorised endpoints reject state mutations at the server level. Data Isolation: Figure 3 illustrates the administrative War Room Alerts dashboard configured for Tier 3 (Compliance Officer) roles and above. The transmission payloads, severity metrics, and risk categorisations are streamed via authenticated sessions that match the active role index.

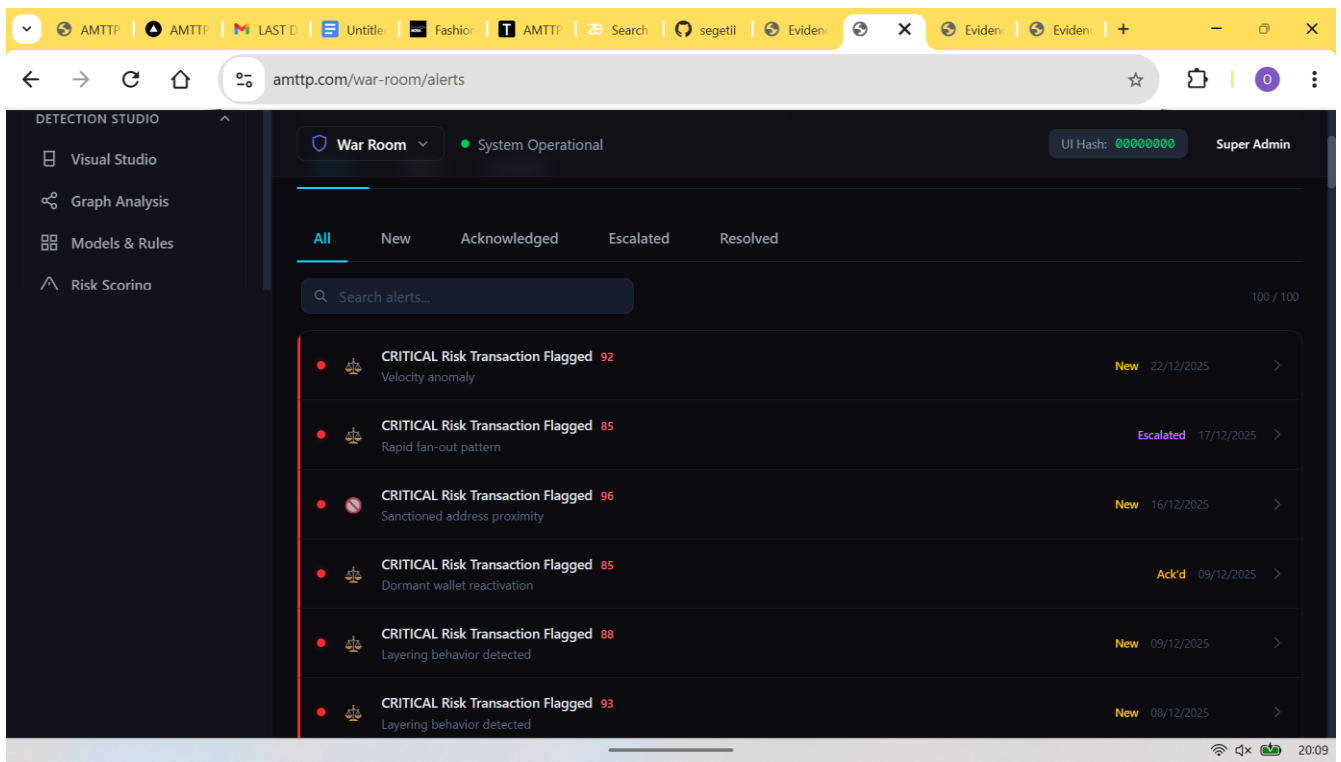


Figure 3: War Room Alerts panel (Compliance Officers and above, enforced server-side) — 100 live critical alerts with severity scores and flag reasons. Role access is denied at the backend, so bypassing the UI restriction gains nothing.

4. Enterprise Transparency & Client Interface Scaling

Decision Audit Transparency: to assist compliance personnel in verifying machine learning classifications, the Explainability Panel (Figure 4) maps real-time feature contributions (e.g., High Risk Transfer weight at 85%, proximity to flagged addresses at 65%). This module translates raw model outputs into an auditable format that satisfies FCA and MiCA operational visibility standards.

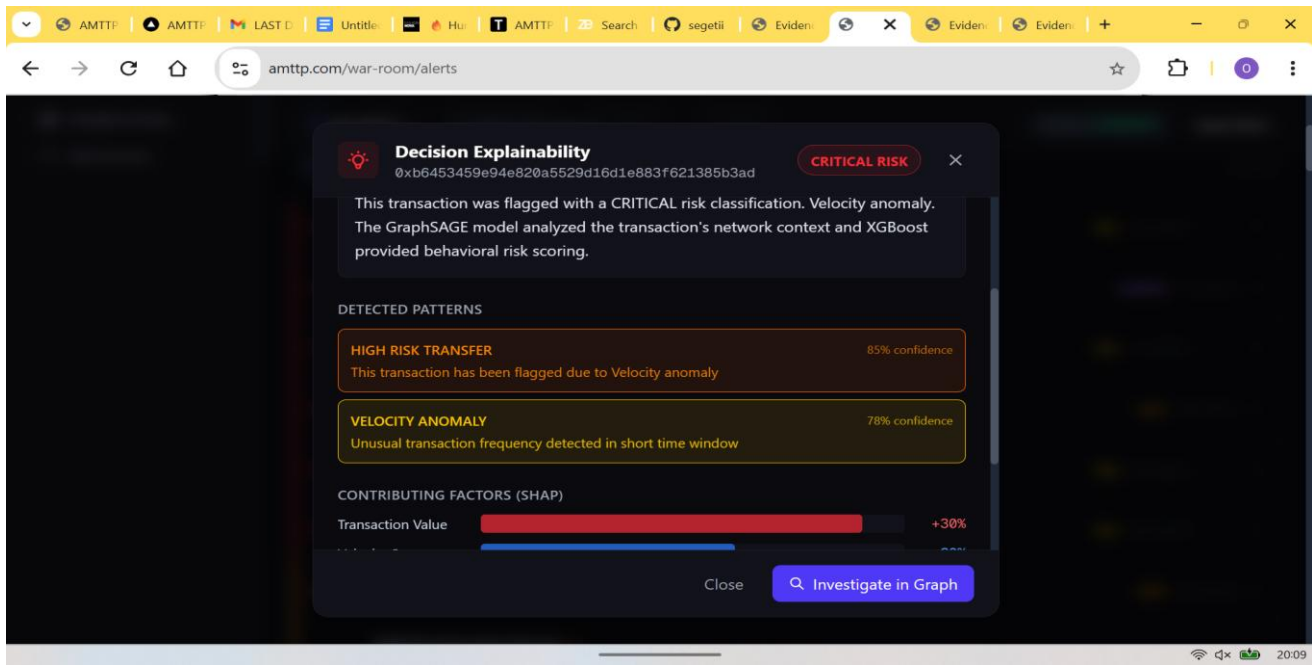


Figure 4: Decision Explainability panel for a CRITICAL alert — contributing factors and their weights: High Risk Transfer 85%, Velocity Anomaly 78%, Sanctioned Address Proximity 65%, with direct link to the graph network view. Satisfies FCA/MiCA audit trail requirements without additional tooling.

5. Cross-Platform Client Scaling

Figures 5 and 6 illustrate the consumer-facing layer of the protocol, developed using the Flutter framework in Dart from a single unified codebase. The mobile client interfaces connect to the identical FastAPI orchestrator and shared RBAC definitions used by the enterprise web dashboard. This guarantees that a single transaction risk verdict maps correctly and identically across the entire consumer and corporate ecosystem. The codebase targets iOS, Android, Web, macOS, Windows, and Linux from a single source — sole-authored.

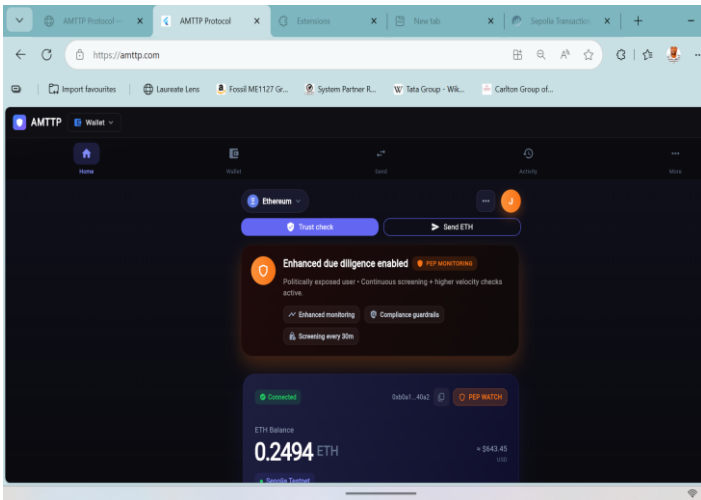


Figure 5: Home screen
(R2 PEP / Enhanced Monitoring)

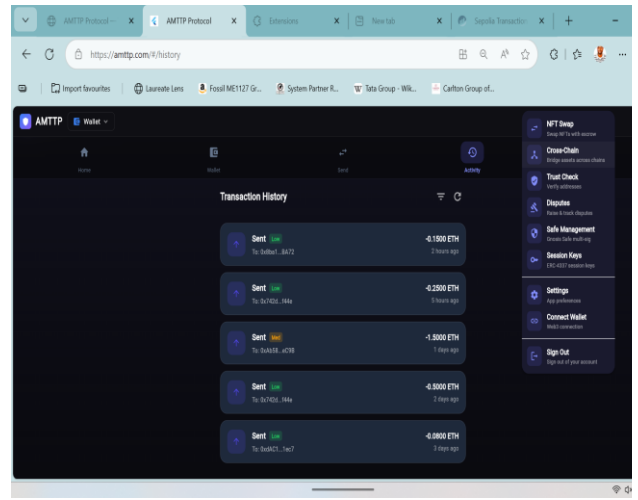


Figure 6: Activity feed
& navigation menu

Unified Backend State: both the War Room (R3–R6) and Flutter Focus Mode (R1–R2) connect to the same backend and the same RBAC configuration. A single compliance verdict is surfaced in a contextually appropriate UI for each role, ensuring architectural consistency across the full user spectrum from individual wallet holders to enterprise compliance teams.

Independent Endorsement (Letter A)

Prof. A. S. O. Ogunjuyigbe (Professor of Electrical Power & Energy Conversion Systems and Provost, Postgraduate College, University of Ibadan) independently reviewed the AMTTP platform and noted: “[The system comprises] a compliance orchestration engine of seven parallel microservices ... The entire stack operates across 17 containerised microservices. Olusegun has designed and implemented this system independently.” (Letter A — full text submitted as supporting reference letter.) Note: Prof. Ogunjuyigbe’s evaluation was performed during the initial protocol deployment phase when 7 compliance microservices were operational. The platform has since expanded to its current production architecture of 9 parallel microservices to accommodate zero-knowledge identity assertions (ZK-NAF, port 8010) and real-time interface integrity hashing (Integrity Service, port 8008), with the full stack operating across 17 containerised microservices.